

Product Packets Generator

Autocoder User Guide

1 Description

The purpose of this generator is to provide a user friendly way of creating packets formed from a list of data products. The generator takes a YAML model file as input which specifies the packets to produce, the data products to put in each packet, the period that the packet will be emitted at, and whether the packet is enabled or disabled on startup. From this information, the generator autocodes an Ada specification file which contains a data structure that should be passed to the Product Packetizer component upon initialization.

Note the example shown in this documentation is used in the unit test of this component so that the reader of this document can see it being used in context. Please refer to the unit test code for more details on how this generator can be used.

2 Schema

The following pykwalify schema is used to validate the input YAML model. Model files must be named in the form *optional_name.assembly_name.product_packets.yaml* where *optional_name* is the specific name of this set of packets and is only necessary if there is more than one Product Packetizer component instance in an assembly. The *assembly_name* is the assembly which these product packets will be used in, and the rest of the model file name must remain as shown. Generally this file is created in the same directory or near to the assembly model file. The schema is commented to show what each of the available YAML keys are and what they accomplish. Even without knowing the specifics of pykwalify schemas, you should be able to glean some knowledge from the file below.

```
1  ---
2  # This schema describes the yaml format for a data product packet suite.
3  type: map
4  mapping:
5    # Description of the packet suite.
6    description:
7      type: str
8      required: False
9    # Many "with" dependencies are automatically deduced and included by
10    # the generator. If you want to manually add a "with" statement, you
11    # can list the names of the packages here.
12    with:
13      seq:
14        - type: str
15        required: False
16    # List of packets to include in the suite.
17    packets:
18      seq:
19        - type: map
20          mapping:
21            # Name of the packet.
22            name:
23              type: str
```

```

24         required: True
25     # Description of the packet.
26     description:
27         type: str
28         required: False
29     # Identifier for the packet (in CCSDS this would be the APID).
30     id:
31         type: int
32         required: True
33     # Is the packet enabled or disabled upon initialization. By default
34     # packets are enabled if this is not specified. This field can be set
35     ↪ to:
36     #     True - Packet is enabled and sent periodically
37     #     False - Packet is disabled
38     #     On_Change - Packet is only sent when data products have changed
39     ↪ since last emission
40     enabled:
41         type: any
42         required: False
43     # The period (in ticks) in which to build the packet. This is the
44     ↪ value set upon
45     # initialization.
46     period:
47         type: str
48         required: True
49     # The offset (in ticks) at which to stagger the construction of this
50     ↪ packet. An offset of
51     # 5 will cause the packet to be built according to its period, but 5
52     ↪ ticks later than expected.
53     # Note that the offset should be less than the period otherwise it
54     ↪ will be mod'd by the period
55     # so that it is less than the period. For example, if the period is 3
56     ↪ and the offset is set to
57     # 5, the actual offset used will be 2.
58     #
59     # This field can be used to stagger packet creation, allowing the user
60     ↪ to evenly distribute
61     # the work that this component does, so as to not cause cycle slips
62     ↪ when many packets need to
63     # be built on the same tick.
64     offset:
65         type: str
66         required: False
67     # If set to true then the packet is timestamped with the time found on
68     ↪ the incoming Tick.T
69     # instead of the current time as fetched via the time connector. By
70     ↪ default, if not specified
71     # this value is set to False.
72     use_tick_timestamp:
73         type: bool
74         required: False
75     # List of data products to include in packet
76     data_products:
77         seq:
78             - type: map
79             mapping:
80                 # The name of the data product. The name should be in the
81                 ↪ format
82                 # Component_Name.Data_Product_Name. The name is a required
83                 ↪ field unless
84                 # pad_bytes is specified. If specified alongside pad_bytes,
85                 ↪ the name

```

```

72         # instead labels the pad bytes in the generated packet layout
73         ↪ and ground
74         # artifacts. A pad name may be a bare identifier or a dotted
75         ↪ path (useful
76         # to mirror the name a future real item will have on the
77         ↪ ground) and is
78         # never resolved against the assembly.
79         name:
80             type: str
81             required: False
82         # Produce an event if the data product is ever not available
83         ↪ when fetched. By default
84         # this is false.
85         event_on_missing:
86             type: bool
87             required: False
88         # Use this data product's timestamp as the packet timestamp.
89         ↪ This may only be set true for
90         # a single data product per packet. By default this value is
91         ↪ false.
92         use_timestamp:
93             type: bool
94             required: False
95         # Include this data product's timestamp just before its value
96         ↪ in the actual packet.
97         # By default this value is false.
98         include_timestamp:
99             type: bool
100             required: False
101         # When the packet's enabled field is set to On_Change, this
102         ↪ field determines whether
103         # this data product's timestamp should be checked to trigger
104         ↪ packet emission. If true,
105         # the packet will be sent if this data product has been
106         ↪ updated since the last packet
107         # emission. If false, this data product's timestamp is ignored
108         ↪ for on_change checks.
109         # By default this value is true if not specified.
110         used_for_on_change:
111             type: bool
112             required: False
113         # Pad bytes can be used to insert a n-number of bytes of
114         ↪ unused data into a packet. This is
115         # also useful to add in spacing for data products that do not
116         ↪ exist yet, but are expected
117         # in the packet. Pad bytes may optionally be combined with
118         ↪ "name" to label the reserved
119         # space in the generated packet layout and ground artifacts.
120         ↪ No other fields may be
121         # combined with pad_bytes.
122         pad_bytes:
123             type: int
124             required: False
125     range:
126         min: 1
127         required: True
128     # A packet suite must have at least one packet.
129     range:
130         min: 1
131         required: True

```

3 Example Input

The following is an example product packet input yaml file. Model files must be named in the form *optional_name.assembly_name.product_packets.yaml* where *optional_name* is the specific name of the product packets and is only necessary if there is more than one Product Packetizer component instance in an assembly. The *assembly_name* is the assembly which these packets will be used in, and the rest of the model file name must remain as shown. Generally this file is created in the same directory or near to the assembly model file. This example adheres to the schema shown in the previous section, and is commented to give clarification.

```
1  ---
2  description: This is an example set of packets.
3  # starting id...
4  # assuming 1 hz tick
5  packets:
6    - name: Packet_1 # must be unique, enforce by autocoder
7      description: This is packet 1.
8      id: 7
9      data_products:
10         - name: Test_Component_1_Instance.Data_Product_A
11           use_timestamp: False
12           include_timestamp: True
13           event_on_missing: True
14         - name: Test_Component_2_Instance.Data_Product_C
15           event_on_missing: False
16           use_timestamp: False
17           include_timestamp: False
18      period: "3" # create every 3 ticks
19      enabled: True
20    - name: Packet_2
21      id: 9
22      data_products:
23         - name: Test_Component_2_Instance.Data_Product_D
24         - name: Test_Component_1_Instance.Data_Product_B
25           use_timestamp: True
26      period: "1" # create every tick
27      offset: "0"
28      enabled: False
29    - name: Packet_3 # must be unique, enforce by autocoder
30      description: This is packet 1.
31      id: 8
32      use_tick_timestamp: False
33      data_products:
34         - name: Test_Component_1_Instance.Data_Product_A
35           use_timestamp: False
36           include_timestamp: True
37           event_on_missing: True
38         - name: Test_Component_2_Instance.Data_Product_C
39           event_on_missing: False
40           use_timestamp: False
41           include_timestamp: False
42      period: "3" # create every 3 ticks
43      offset: "5" # This should act like an offset of 2, but we are testing that
44        ↪ feature here.
45      enabled: False
46    - name: Packet_4
47      description: This packet tests padding
48      id: 12
49      use_tick_timestamp: True
50      data_products:
51         - pad_bytes: 5
```

```

51     - name: Test_Component_1_Instance.Data_Product_A
52       use_timestamp: False
53       include_timestamp: True # Include a timestamp mid-packet, after a pad.
54       event_on_missing: False
55     - pad_bytes: 3
56     - pad_bytes: 1
57   period: "1" # create every tick
58   offset: "0"
59   enabled: False
60 - name: Packet_5
61   id: 15
62   data_products:
63     - name: Product_Packetizer_Instance.Packet_4_Period
64     - name: Product_Packetizer_Instance.Packet_5_Period
65     - name: Product_Packetizer_Instance.Packet_3_Period
66   period: "2"
67   enabled: False
68 - name: Packet_6
69   description: This packet tests named padding
70   id: 16
71   use_tick_timestamp: True
72   data_products:
73     - name: Omeg_Norm # A named pad with a bare identifier name.
74       pad_bytes: 8
75     - name: Test_Component_1_Instance.Future_Item # A named pad with a dotted
76       ↪ name.
77       pad_bytes: 4
78     - pad_bytes: 2
79     - pad_bytes: 1
80   period: "1" # create every tick
81   offset: "0"
82   enabled: False

```

4 Example Output

The example input shown in the previous section produces the following Ada output. The `Packet_List` variable should be passed into the Product Packetizer component's discriminant during assembly initialization.

The main job of the generator in this case was to verify the input YAML packets for validity and then to translate the data to an Ada data structure for use by the component.

```

1  -- Standard includes:
2  with Product_Packet_Types; use Product_Packet_Types;
3  with Packet_Types;
4  with Sys_Time.Arithmetic;
5
6  -- This is an example set of packets.
7  package Test_Assembly_Product_Packets_Test_Packets is
8
9      -- Packet_1:
10     -- This is packet 1.
11
12     -- Packet_1 data product items:
13     -- Total packet buffer size: 160 bits
14     Packet_1_Items : aliased Packet_Items_Type := [
15         -- Item entry for Test_Component_1_Instance.Data_Product_A:
16         1 => (Data_Product_Id => 1, Use_Timestamp => False, Include_Timestamp =>
17             ↪ True, Event_On_Missing => True, Used_For_On_Change => True,
18             ↪ Packet_Period_Item => False, Size => 4),

```

```

17      -- Item entry for Test_Component_2_Instance.Data_Product_C:
18      2 => (Data_Product_Id => 3, Use_Timestamp => False, Include_Timestamp =>
      ↪ False, Event_On_Missing => False, Used_For_On_Change => True,
      ↪ Packet_Period_Item => False, Size => 10)
19  ];
20
21  -- Packet_1 packet description:
22  Packet_1_Description : Packet_Description_Type := (
23      Id => 7,
24      Items => Packet_1_Items'Access,
25      Period => 3,
26      Offset => 0,
27      Enabled => Product_Packet_Types.Enabled,
28      Use_Tick_Timestamp => False,
29      Count => Packet_Types.Sequence_Count_Mod_Type'First,
30      Send_Now => False,
31      Last_Emission_Time => Sys_Time.Arithmetic.Sys_Time_Zero
32  );
33
34  -- Packet_2:
35
36  -- Packet_2 data product items:
37  -- Total packet buffer size: 96 bits
38  Packet_2_Items : aliased Packet_Items_Type := [
39      -- Item entry for Test_Component_2_Instance.Data_Product_D:
40      1 => (Data_Product_Id => 4, Use_Timestamp => False, Include_Timestamp =>
      ↪ False, Event_On_Missing => False, Used_For_On_Change => True,
      ↪ Packet_Period_Item => False, Size => 2),
41      -- Item entry for Test_Component_1_Instance.Data_Product_B:
42      2 => (Data_Product_Id => 2, Use_Timestamp => True, Include_Timestamp =>
      ↪ False, Event_On_Missing => False, Used_For_On_Change => True,
      ↪ Packet_Period_Item => False, Size => 10)
43  ];
44
45  -- Packet_2 packet description:
46  Packet_2_Description : Packet_Description_Type := (
47      Id => 9,
48      Items => Packet_2_Items'Access,
49      Period => 1,
50      Offset => 0,
51      Enabled => Product_Packet_Types.Disabled,
52      Use_Tick_Timestamp => False,
53      Count => Packet_Types.Sequence_Count_Mod_Type'First,
54      Send_Now => False,
55      Last_Emission_Time => Sys_Time.Arithmetic.Sys_Time_Zero
56  );
57
58  -- Packet_3:
59  -- This is packet 1.
60
61  -- Packet_3 data product items:
62  -- Total packet buffer size: 160 bits
63  Packet_3_Items : aliased Packet_Items_Type := [
64      -- Item entry for Test_Component_1_Instance.Data_Product_A:
65      1 => (Data_Product_Id => 1, Use_Timestamp => False, Include_Timestamp =>
      ↪ True, Event_On_Missing => True, Used_For_On_Change => True,
      ↪ Packet_Period_Item => False, Size => 4),
66      -- Item entry for Test_Component_2_Instance.Data_Product_C:
67      2 => (Data_Product_Id => 3, Use_Timestamp => False, Include_Timestamp =>
      ↪ False, Event_On_Missing => False, Used_For_On_Change => True,
      ↪ Packet_Period_Item => False, Size => 10)

```

```

68 ];
69
70 -- Packet_3 packet description:
71 Packet_3_Description : Packet_Description_Type := (
72     Id => 8,
73     Items => Packet_3_Items'Access,
74     Period => 3,
75     Offset => 5,
76     Enabled => Product_Packet_Types.Disabled,
77     Use_Tick_Timestamp => False,
78     Count => Packet_Types.Sequence_Count_Mod_Type'First,
79     Send_Now => False,
80     Last_Emission_Time => Sys_Time.Arithmetic.Sys_Time_Zero
81 );
82
83 -- Packet_4:
84 -- This packet tests padding
85
86 -- Packet_4 data product items:
87 -- Total packet buffer size: 152 bits
88 Packet_4_Items : aliased Packet_Items_Type := [
89     -- Item entry for pad bytes:
90     1 => (Data_Product_Id => 0, Use_Timestamp => False, Include_Timestamp =>
91         ↪ False, Event_On_Missing => False, Used_For_On_Change => True,
92         ↪ Packet_Period_Item => False, Size => 5),
93     -- Item entry for Test_Component_1_Instance.Data_Product_A:
94     2 => (Data_Product_Id => 1, Use_Timestamp => False, Include_Timestamp =>
95         ↪ True, Event_On_Missing => False, Used_For_On_Change => True,
96         ↪ Packet_Period_Item => False, Size => 4),
97     -- Item entry for pad bytes:
98     3 => (Data_Product_Id => 0, Use_Timestamp => False, Include_Timestamp =>
99         ↪ False, Event_On_Missing => False, Used_For_On_Change => True,
100        ↪ Packet_Period_Item => False, Size => 3),
101     -- Item entry for pad bytes:
102     4 => (Data_Product_Id => 0, Use_Timestamp => False, Include_Timestamp =>
103        ↪ False, Event_On_Missing => False, Used_For_On_Change => True,
104        ↪ Packet_Period_Item => False, Size => 1)
105 ];
106
107 -- Packet_4 packet description:
108 Packet_4_Description : Packet_Description_Type := (
109     Id => 12,
110     Items => Packet_4_Items'Access,
111     Period => 1,
112     Offset => 0,
113     Enabled => Product_Packet_Types.Disabled,
114     Use_Tick_Timestamp => True,
115     Count => Packet_Types.Sequence_Count_Mod_Type'First,
116     Send_Now => False,
117     Last_Emission_Time => Sys_Time.Arithmetic.Sys_Time_Zero
118 );
119
120 -- Packet_5:
121
122 -- Packet_5 data product items:
123 -- Total packet buffer size: 96 bits
124 Packet_5_Items : aliased Packet_Items_Type := [
125     -- Item entry for Product_Packetizer_Instance.Packet_4_Period:
126     1 => (Data_Product_Id => 4, Use_Timestamp => False, Include_Timestamp =>
127         ↪ False, Event_On_Missing => False, Used_For_On_Change => True,
128         ↪ Packet_Period_Item => True, Size => 4),

```

```

119      -- Item entry for Product_Packetizer_Instance.Packet_5_Period:
120      2 => (Data_Product_Id => 5, Use_Timestamp => False, Include_Timestamp =>
        ↳ False, Event_On_Missing => False, Used_For_On_Change => True,
        ↳ Packet_Period_Item => True, Size => 4),
121      -- Item entry for Product_Packetizer_Instance.Packet_3_Period:
122      3 => (Data_Product_Id => 3, Use_Timestamp => False, Include_Timestamp =>
        ↳ False, Event_On_Missing => False, Used_For_On_Change => True,
        ↳ Packet_Period_Item => True, Size => 4)
123  ];
124
125  -- Packet_5 packet description:
126  Packet_5_Description : Packet_Description_Type := (
127      Id => 15,
128      Items => Packet_5_Items'Access,
129      Period => 2,
130      Offset => 0,
131      Enabled => Product_Packet_Types.Disabled,
132      Use_Tick_Timestamp => False,
133      Count => Packet_Types.Sequence_Count_Mod_Type'First,
134      Send_Now => False,
135      Last_Emission_Time => Sys_Time.Arithmetic.Sys_Time_Zero
136  );
137
138  -- Packet_6:
139  -- This packet tests named padding
140
141  -- Packet_6 data product items:
142  -- Total packet buffer size: 120 bits
143  Packet_6_Items : aliased Packet_Items_Type := [
144      -- Item entry for Omeg_Norm:
145      1 => (Data_Product_Id => 0, Use_Timestamp => False, Include_Timestamp =>
        ↳ False, Event_On_Missing => False, Used_For_On_Change => True,
        ↳ Packet_Period_Item => False, Size => 8),
146      -- Item entry for Test_Component_1_Instance.Future_Item:
147      2 => (Data_Product_Id => 0, Use_Timestamp => False, Include_Timestamp =>
        ↳ False, Event_On_Missing => False, Used_For_On_Change => True,
        ↳ Packet_Period_Item => False, Size => 4),
148      -- Item entry for pad bytes:
149      3 => (Data_Product_Id => 0, Use_Timestamp => False, Include_Timestamp =>
        ↳ False, Event_On_Missing => False, Used_For_On_Change => True,
        ↳ Packet_Period_Item => False, Size => 2),
150      -- Item entry for pad bytes:
151      4 => (Data_Product_Id => 0, Use_Timestamp => False, Include_Timestamp =>
        ↳ False, Event_On_Missing => False, Used_For_On_Change => True,
        ↳ Packet_Period_Item => False, Size => 1)
152  ];
153
154  -- Packet_6 packet description:
155  Packet_6_Description : Packet_Description_Type := (
156      Id => 16,
157      Items => Packet_6_Items'Access,
158      Period => 1,
159      Offset => 0,
160      Enabled => Product_Packet_Types.Disabled,
161      Use_Tick_Timestamp => True,
162      Count => Packet_Types.Sequence_Count_Mod_Type'First,
163      Send_Now => False,
164      Last_Emission_Time => Sys_Time.Arithmetic.Sys_Time_Zero
165  );
166
167  -- List of packets for the packetizer to build:

```



```

168     Packet_List : aliased Packet_Description_List_Type := [
169         1 => Packet_1_Description,
170         2 => Packet_2_Description,
171         3 => Packet_3_Description,
172         4 => Packet_4_Description,
173         5 => Packet_5_Description,
174         6 => Packet_6_Description
175     ];
176
177 end Test_Assembly_Product_Packets_Test_Packets;

```

5 On-Change Packet Example

The Product Packetizer also supports “on-change” packets that emit only when one or more tracked data products have fresh timestamps relative to the last emission. The enabled enumerual value `On_Change` puts a packet into this mode, while the optional `used_for_on_change` flag attached to each item allows you to include data products that do not participate in the change detection. Any data product with `used_for_on_change = True` (the default) will cause the packet to fire once its timestamp advances; data products tagged with `False` are still copied into the packet but will not on their own schedule a send. Note that the configured packet period still matters—the component evaluates on-change packets on those interval boundaries, but suppresses the transmission if no tracked data product reports a timestamp newer than the last emission.

The dedicated on-change unit tests under `src/components/product_packetizer/test_on_change` exercise this behavior in isolation. The YAML below matches the model compiled in those tests and is intentionally minimal so it can double as documentation for configuring the feature.

5.1 On-Change YAML Model

```

1  ---
2  description: This is an example set of packets.
3  # starting id...
4  # assuming 1 hz tick
5  packets:
6  - name: Packet_1 # must be unique, enforce by autocoder
7    description: Baseline periodic packet used by the on-change tests for
8      ↳ reference behavior.
9    id: 7
10   data_products:
11     - name: Test_Component_1_Instance.Data_Product_A
12       use_timestamp: False
13       include_timestamp: True
14       event_on_missing: True
15     - name: Test_Component_2_Instance.Data_Product_C
16       event_on_missing: False
17       use_timestamp: False
18       include_timestamp: False
19   period: "3" # create every 3 ticks
20   enabled: False
21 - name: Packet_6
22   description: Minimal packet dedicated to validating the On_Change feature.
23   id: 16
24   data_products:
25     - name: Test_Component_2_Instance.Data_Product_D
26       used_for_on_change: True # When True, this data product's timestamp
27         ↳ drives the on-change decision.
28     - name: Test_Component_1_Instance.Data_Product_B

```

```

27     used_for_on_change: False # When False, the data is included but does
    ↪ not trigger on-change sends.
28     period: "1"
29     enabled: On_Change # Packets configured this way emit only when a tracked
    ↪ data product timestamp advances.

```

5.2 Generated Output

Just like the periodic example, the generator produces an Ada structure that is supplied to the Product Packetizer instance. The output derived from the on-change YAML is shown below.

```

1  -- Standard includes:
2  with Product_Packet_Types; use Product_Packet_Types;
3  with Packet_Types;
4  with Sys_Time.Arithmetic;
5
6  -- This is an example set of packets.
7  package Test_Assembly_Product_Packets_Test_Packets is
8
9      -- Packet_1:
10     -- Baseline periodic packet used by the on-change tests for reference
    ↪ behavior.
11
12     -- Packet_1 data product items:
13     -- Total packet buffer size: 160 bits
14     Packet_1_Items : aliased Packet_Items_Type := [
15         -- Item entry for Test_Component_1_Instance.Data_Product_A:
16         1 => (Data_Product_Id => 1, Use_Timestamp => False, Include_Timestamp =>
    ↪ True, Event_On_Missing => True, Used_For_On_Change => True,
    ↪ Packet_Period_Item => False, Size => 4),
17         -- Item entry for Test_Component_2_Instance.Data_Product_C:
18         2 => (Data_Product_Id => 3, Use_Timestamp => False, Include_Timestamp =>
    ↪ False, Event_On_Missing => False, Used_For_On_Change => True,
    ↪ Packet_Period_Item => False, Size => 10)
19     ];
20
21     -- Packet_1 packet description:
22     Packet_1_Description : Packet_Description_Type := (
23         Id => 7,
24         Items => Packet_1_Items'Access,
25         Period => 3,
26         Offset => 0,
27         Enabled => Product_Packet_Types.Disabled,
28         Use_Tick_Timestamp => False,
29         Count => Packet_Types.Sequence_Count_Mod_Type'First,
30         Send_Now => False,
31         Last_Emission_Time => Sys_Time.Arithmetic.Sys_Time_Zero
32     );
33
34     -- Packet_6:
35     -- Minimal packet dedicated to validating the On_Change feature.
36
37     -- Packet_6 data product items:
38     -- Total packet buffer size: 96 bits
39     Packet_6_Items : aliased Packet_Items_Type := [
40         -- Item entry for Test_Component_2_Instance.Data_Product_D:
41         1 => (Data_Product_Id => 4, Use_Timestamp => False, Include_Timestamp =>
    ↪ False, Event_On_Missing => False, Used_For_On_Change => True,
    ↪ Packet_Period_Item => False, Size => 2),
42         -- Item entry for Test_Component_1_Instance.Data_Product_B:

```

```

43     2 => (Data_Product_Id => 2, Use_Timestamp => False, Include_Timestamp =>
    ↪ False, Event_On_Missing => False, Used_For_On_Change => False,
    ↪ Packet_Period_Item => False, Size => 10)
44 ];
45
46 -- Packet_6 packet description:
47 Packet_6_Description : Packet_Description_Type := (
48     Id => 16,
49     Items => Packet_6_Items'Access,
50     Period => 1,
51     Offset => 0,
52     Enabled => Product_Packet_Types.On_Change,
53     Use_Tick_Timestamp => False,
54     Count => Packet_Types.Sequence_Count_Mod_Type'First,
55     Send_Now => False,
56     Last_Emission_Time => Sys_Time.Arithmetic.Sys_Time_Zero
57 );
58
59 -- List of packets for the packetizer to build:
60 Packet_List : aliased Packet_Description_List_Type := [
61     1 => Packet_1_Description,
62     2 => Packet_6_Description
63 ];
64
65 end Test_Assembly_Product_Packets_Test_Packets;

```

6 Special Items

The Product Packetizer allows you to specify “special” items to include in a packet that reflect internal data of the Product Packetizer component itself. Currently, the only supported “special” items are packet periods of the packets produced by the Product Packetizer. Packet 5, specified above, includes these items by specifying a data product within the Product Packetizer, ie. `Product_Packetizer_Instance.Packet_4_Period`. The Product Packetizer doesn’t actually have any data products, so this nomenclature instead denotes a special item. In this case, we want to include the current packet period value (a 4 byte unsigned integer) for Packet 4 into the packet. A period can be specified for any packet included in the YAML model using this pattern. Error checking at the modeling level will prevent you from specifying a packet period for a packet that does not exist.